Reporte Técnico de Actividades Práctico-Experimentales Nro. 006

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Royel Ivan Jima Pardo
Asignatura	Simulación
Ciclo	5 A
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	006
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov
Nombre del Docente	José O. Guamán Q
Fecha	05 de noviembre de 2026 06 de noviembre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Evaluar la calidad estadística de una secuencia de números pseudoaleatorios mediante pruebas de bondad de ajuste, utilizando los métodos Chi-cuadrado y Kolmogorov-Smirnov para determinar si los valores generados presentan un comportamiento compatible con una distribución uniforme en el intervalo [0,1].

1. Generar secuencias de números pseudoaleatorios utilizando el método congruencial lineal (LCG).
2. Aplicar la prueba estadística Chi-cuadrado para evaluar la uniformidad de las frecuencias observadas.
3. Aplicar la prueba Kolmogorov-Smirnov para comparar la distribución empírica con la distribución uniforme teórica.
4. Interpretar los estadísticos de prueba y sus respectivos valores p.
5. Comparar el comportamiento estadístico de diferentes configuraciones del generador pseudoaleatorio

3. Materiales, Reactivos

- Computador.
- Lenguaje de programación Python.
- Lenguaje de programación Java.
- Entorno de desarrollo integrado.
- Generador de números pseudoaleatorios..

4. Equipos y Herramientas

- Python.
- NumPy.
- Matplotlib.
- Pandas.
- JDK.
- IntelliJ IDEA, Eclipse o NetBeans.
- Visual Studio Code

5. Procedimiento / Metodología Ejecutada

- Se revisaron las diapositivas de la semana 6 y se investigó el funcionamiento del generador congruencial lineal (LCG) y de las pruebas estadísticas Chi-cuadrado y Kolmogorov-Smirnov.
- Se estudiaron las fórmulas matemáticas de generación de números pseudoaleatorios, frecuencias esperadas, estadístico Chi-cuadrado y diferencias de distribución acumulada KS.
- Se desarrollaron dos implementaciones independientes utilizando los mismos parámetros del generador LCG, con el propósito de obtener secuencias equivalentes.
- Se incorporaron las pruebas Chi-cuadrado y Kolmogorov-Smirnov para evaluar la uniformidad de los números generados, utilizando un nivel de significancia de 0.05.
- Se realizaron pruebas de funcionamiento, se identificaron y corrigieron errores en los algoritmos y se verificó que ambos lenguajes produjeran resultados equivalentes.
- Se implementaron menús interactivos que permiten seleccionar Python, Java o ambas implementaciones, configurar parámetros y ejecutar las pruebas estadísticas.
- Se ejecutaron diferentes tamaños de muestras (1 000, 10 000, 100 000 y 1 000 000), comparando los estadísticos, valores p y tiempos de ejecución.
- Finalmente, se interpretaron los resultados obtenidos, se elaboraron tablas comparativas y se formularon las conclusiones de la práctica.

6. Resultados

A. Histograma de frecuencias de Python y Java

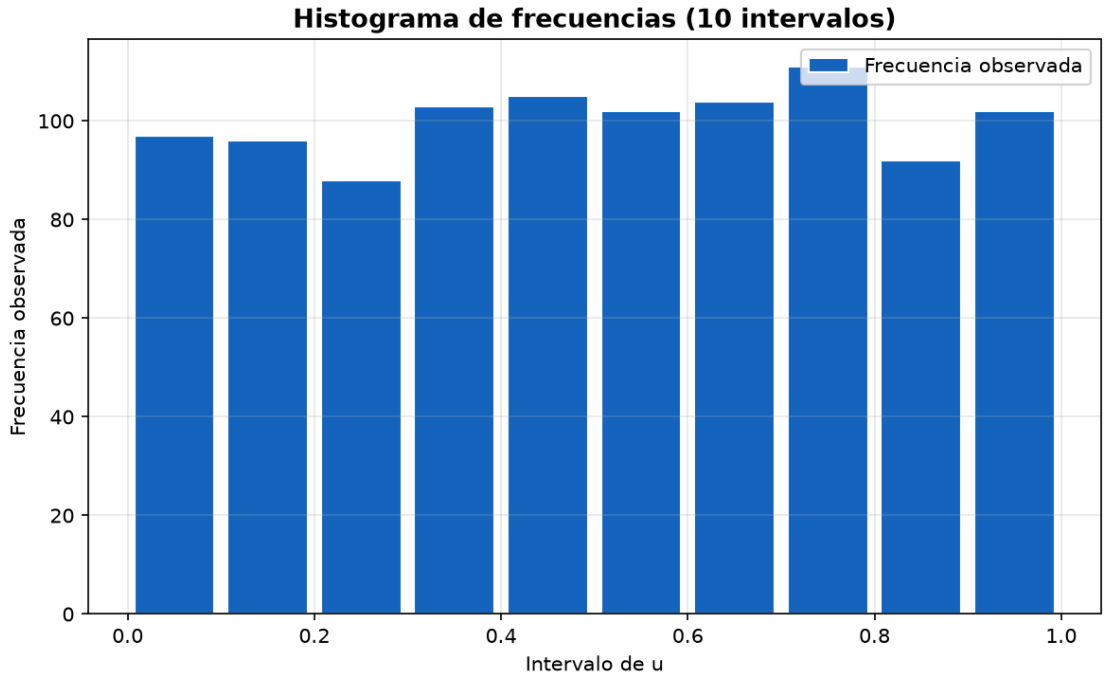


Figura 1. Histograma de frecuencias de Python.

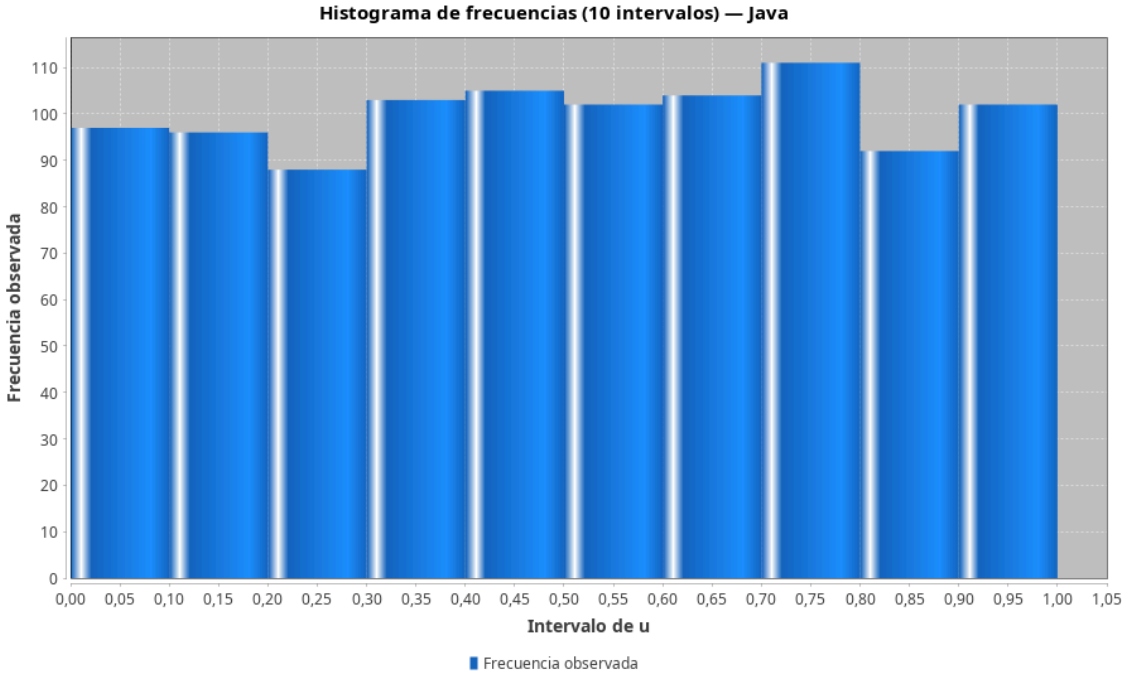


Figura 2. Histograma de frecuencias de Java.

La figura 1 y la figura 2, muestran la distribución de frecuencias de los números pseudoaleatorios generados mediante el método congruencial lineal en diez intervalos. Se observa cómo se distribuyen los valores generados y qué tan cercanas son las frecuencias observadas a las frecuencias esperadas. Una distribución aproximadamente uniforme presenta frecuencias similares entre los intervalos, aunque pueden existir pequeñas variaciones aleatorias.

B. Comparación ECDF vs. distribución uniforme teórica

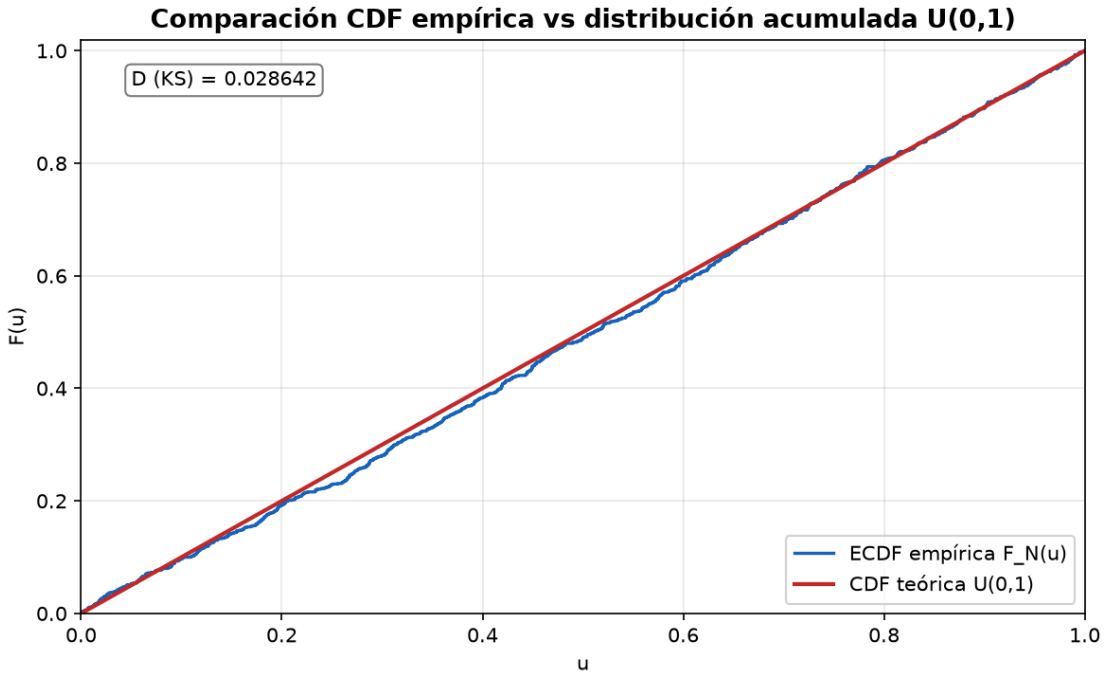


Figura 3. ECDF vs. uniforme teórica de Python.

En la figura 3, se compara la función de distribución acumulada empírica (ECDF) de los números pseudoaleatorios con la función de distribución acumulada uniforme teórica. La línea empírica representa el comportamiento acumulado de los datos generados, mientras que la línea teórica representa el comportamiento esperado para una distribución uniforme entre 0 y 1. Cuanto más próximas se encuentren ambas curvas, menor será la diferencia entre la distribución observada y la teórica. Esta comparación permite visualizar el comportamiento evaluado mediante la prueba de Kolmogorov-Smirnov.

C. Tabla de resultados estadísticos

Tabla 1. Resultados de las pruebas estadísticas de uniformidad en Python y Java

Lenguaje	N	Chi-cuadrado (χ^2)	p-valor Chi²	KS (D)	p-valor KS	Decisión
Python / Java	1.000	4.1200	0.903338	0.028642	0.377741	No se rechaza H_0
Python / Java	10.000	14.1600	0.116746	0.005960	0.867344	No se rechaza H_0
Python / Java	100.000	9.1600	0.422638	0.002073	0.782460	No se rechaza H_0
Python / Java	1.000.000	16.0227	0.066409	0.000944	0.334555	No se rechaza H_0

Los resultados de Python y Java coinciden para todos los tamaños de muestra. Se utilizó un nivel de significancia $\alpha = 0.05$.

Tabla 2. Comparación de tiempos de ejecución entre Python y Java

Cantidad de números (N)	Tiempo Python (ms)	Tiempo Java (ms)	Lenguaje más rápido
1.000	4.1	4.4	Python
10.000	17.0	20.8	Python
100.000	200.8	54.4	Java
1.000.000	3878.7	475.9	Java

Los tiempos corresponden al promedio de tres ejecuciones, después de una ejecución de calentamiento que no fue contabilizada. No incluyen la carga de la JVM ni la generación de gráficas.

Análisis de los resultados

Los resultados obtenidos muestran que las implementaciones desarrolladas en Python y Java generaron secuencias idénticas de números pseudoaleatorios para los cuatro tamaños de muestra evaluados: 1.000, 10.000, 100.000 y 1.000.000. Esta coincidencia confirma que ambos programas utilizan correctamente los mismos parámetros del generador congruencial lineal.

En la prueba Chi-cuadrado, los valores p obtenidos fueron superiores al nivel de significancia de 0.05 en todos los casos. De igual manera, en la prueba Kolmogorov-Smirnov, los valores p también fueron superiores a dicho nivel. Por lo tanto, no se rechaza la hipótesis nula (H_0), lo que indica que no se encontró evidencia estadística suficiente para afirmar que las secuencias generadas no siguen una distribución uniforme.

En cuanto al rendimiento, Python presentó menores tiempos de ejecución para las muestras de 1.000 y 10.000 números. Sin embargo, Java obtuvo mejores tiempos para las muestras de 100.000 y 1.000.000 números, mostrando una ventaja de rendimiento a medida que aumenta el tamaño de la muestra.

Finalmente, se comprobó que ambas implementaciones producen los mismos resultados estadísticos y las mismas secuencias, aunque presentan diferencias en sus tiempos de ejecución.

7. Preguntas de Control

1. ¿Qué es la hipótesis nula?

La hipótesis nula (H_0) es una afirmación que establece que no existe una diferencia significativa entre los resultados observados y los esperados. En esta práctica, plantea que los números pseudoaleatorios generados siguen una distribución uniforme en el intervalo [0,1].

2. ¿Qué representa el valor p?

El valor p representa la probabilidad, suponiendo que la hipótesis nula sea verdadera, de obtener un resultado igual o más extremo que el observado. Permite determinar si existe evidencia estadística suficiente para rechazar H_0.

3. ¿Qué significa rechazar H_0?

Significa que existe evidencia estadística suficiente para considerar que los datos no se ajustan a la distribución uniforme esperada. En esta práctica, se rechaza H_0 cuando el valor p es menor que el nivel de significancia de 0.05.

4. ¿Qué diferencia existe entre Chi-cuadrado y KS?

La prueba Chi-cuadrado compara las frecuencias observadas con las frecuencias esperadas en diferentes intervalos. En cambio, Kolmogorov-Smirnov compara la distribución acumulada empírica de los datos con la distribución acumulada teórica, identificando la mayor diferencia entre ambas.

5. ¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?

Porque las pruebas estadísticas únicamente evalúan determinadas propiedades de una secuencia, como la uniformidad, pero no demuestran que sea verdaderamente aleatoria. Un generador pseudoaleatorio es determinista, ya que produce resultados a partir de una semilla y un algoritmo definido. Por lo tanto, puede superar ciertas pruebas estadísticas sin ser realmente aleatorio.

8. Conclusiones

- Se generaron satisfactoriamente secuencias de números pseudoaleatorios mediante el método congruencial lineal (LCG), utilizando los mismos parámetros en Python y Java. Se comprobó que ambas implementaciones produjeron secuencias exactamente iguales para los tamaños de muestra de 1.000, 10.000, 100.000 y 1.000.000 números, demostrando la consistencia de las implementaciones.
- Mediante la prueba Chi-cuadrado se evaluó la uniformidad de las frecuencias observadas en diez intervalos. Los valores p obtenidos fueron 0.903338, 0.116746, 0.422638 y 0.066409, respectivamente. Debido a que todos fueron superiores al nivel de significancia de 0.05, no se rechazó la hipótesis nula, por lo que no se encontró evidencia estadística suficiente para afirmar que las secuencias no siguen una distribución uniforme.
- La interpretación de los estadísticos y sus respectivos valores p permitió determinar que las secuencias generadas presentan un comportamiento compatible con una distribución uniforme en el intervalo [0,1], de acuerdo con las pruebas aplicadas y el nivel de significancia establecido. Sin embargo, estos resultados no demuestran que los números sean verdaderamente aleatorios, sino que no se encontró evidencia suficiente para rechazar su uniformidad.
- La comparación de los resultados para diferentes tamaños de muestra permitió observar que ambas implementaciones conservaron los mismos resultados estadísticos y generaron secuencias idénticas.

9. Anexos

Diapositivas de la semana 6